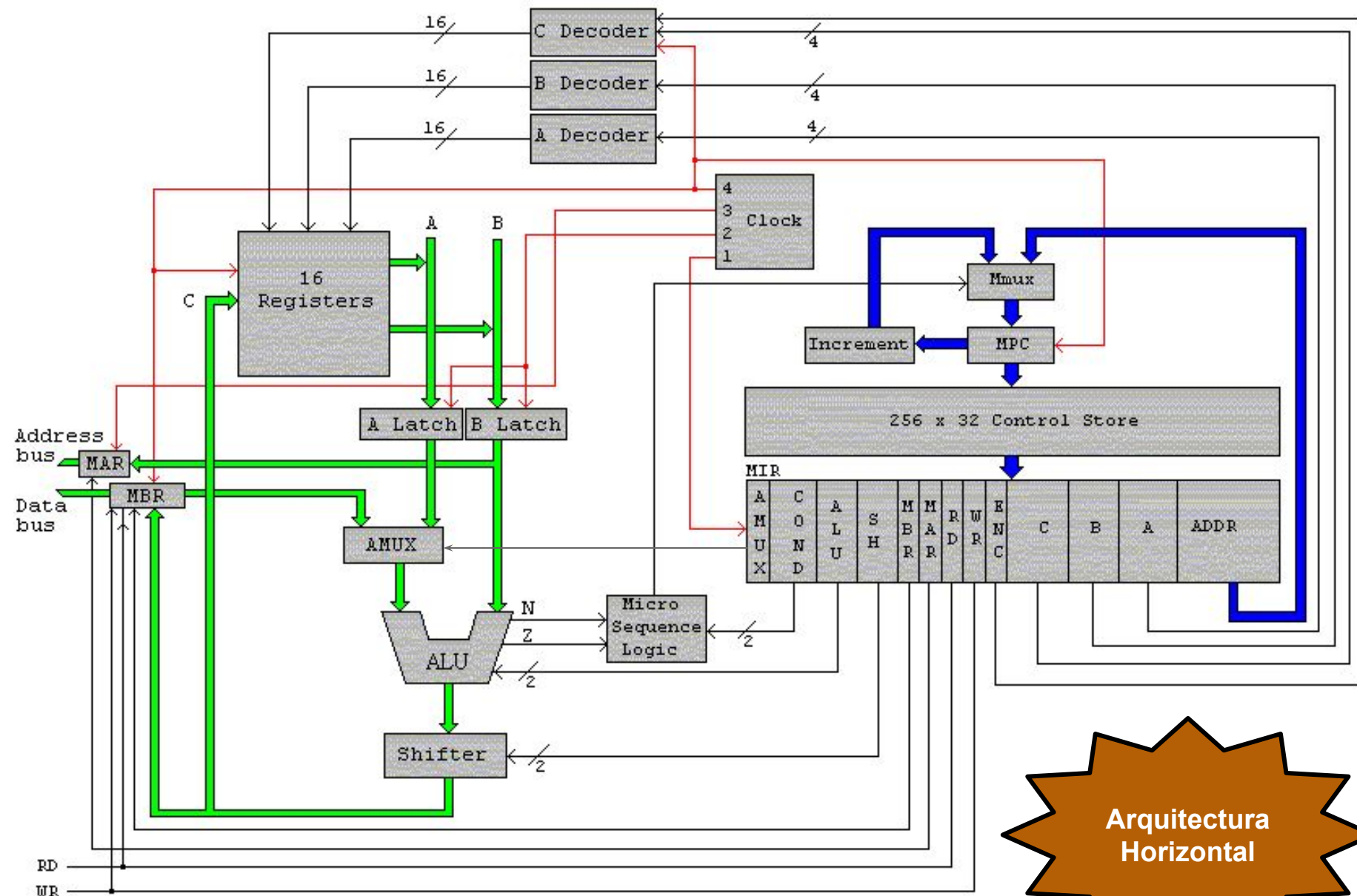


Micro Arquitectura Vertical

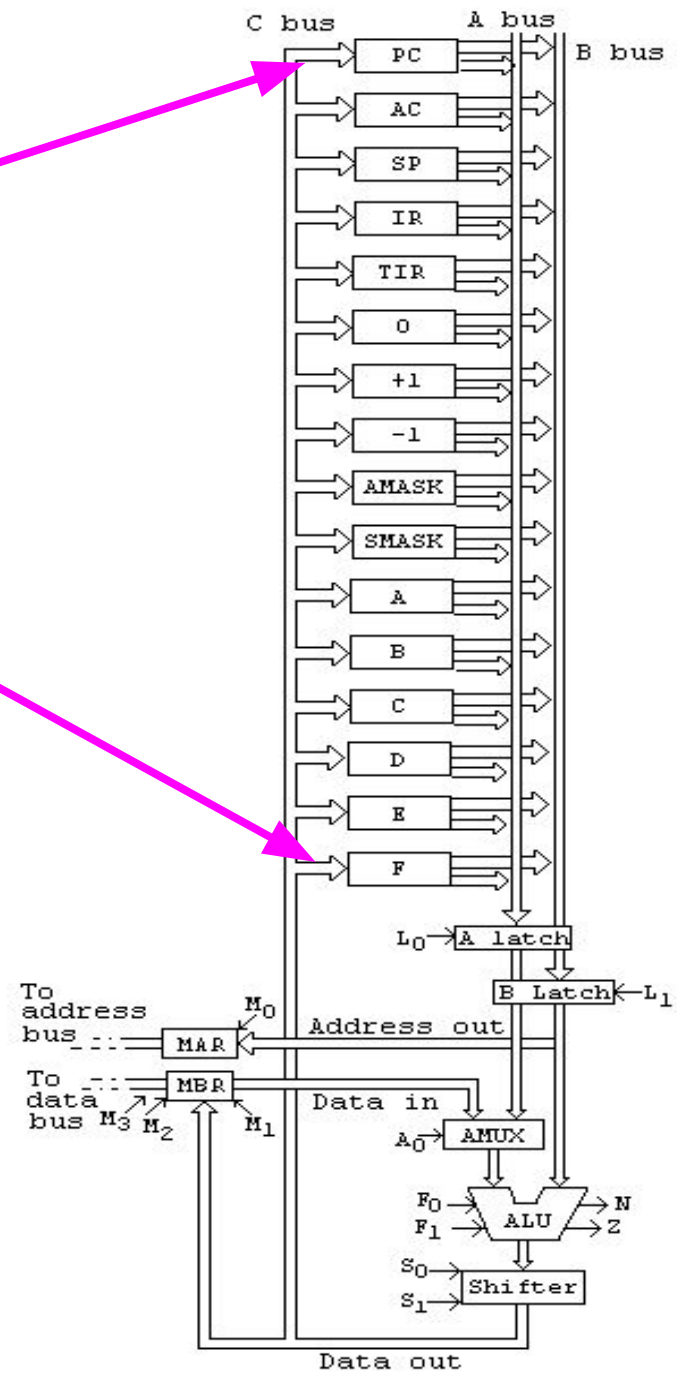
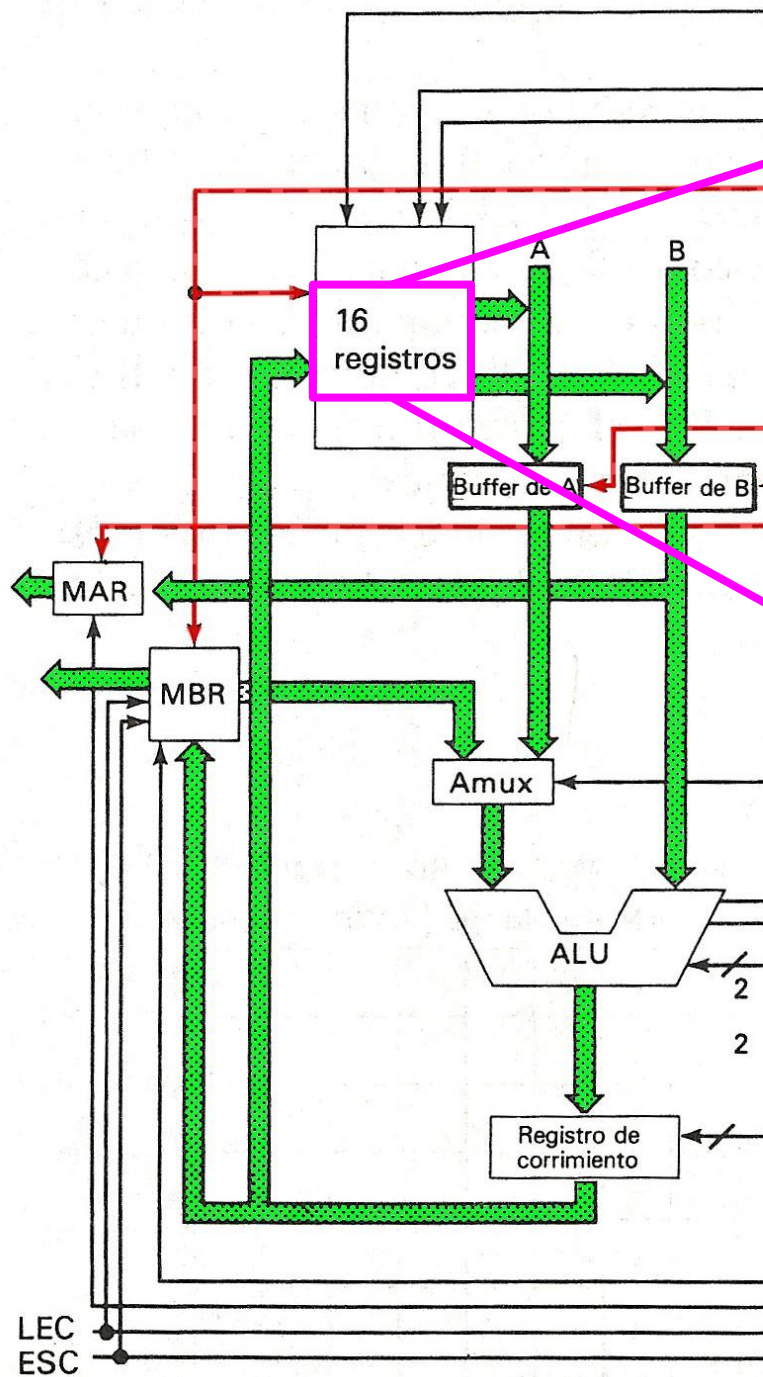
Nano-Programación y Pipe-Line

Arquitectura de computadoras

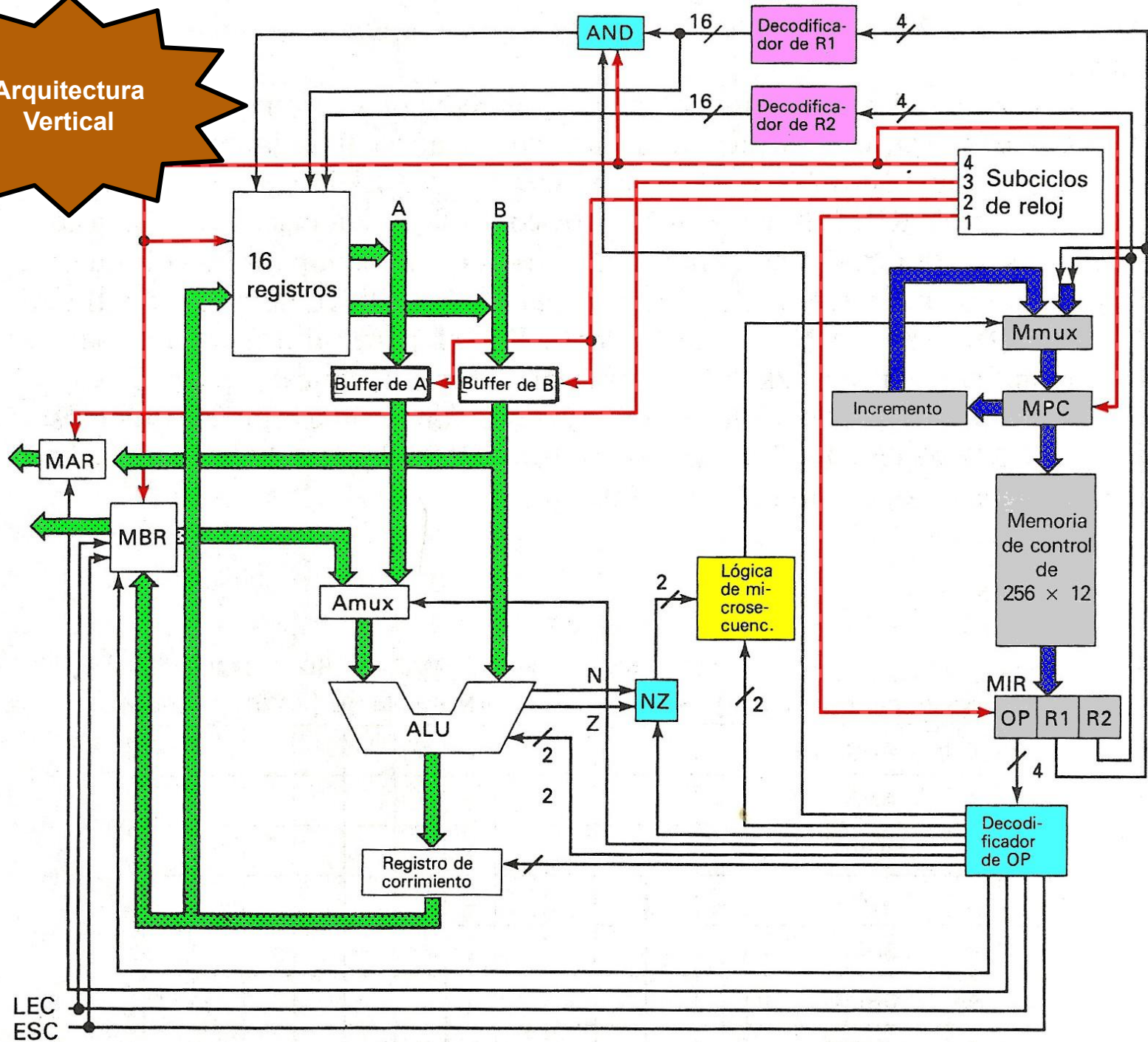
Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta



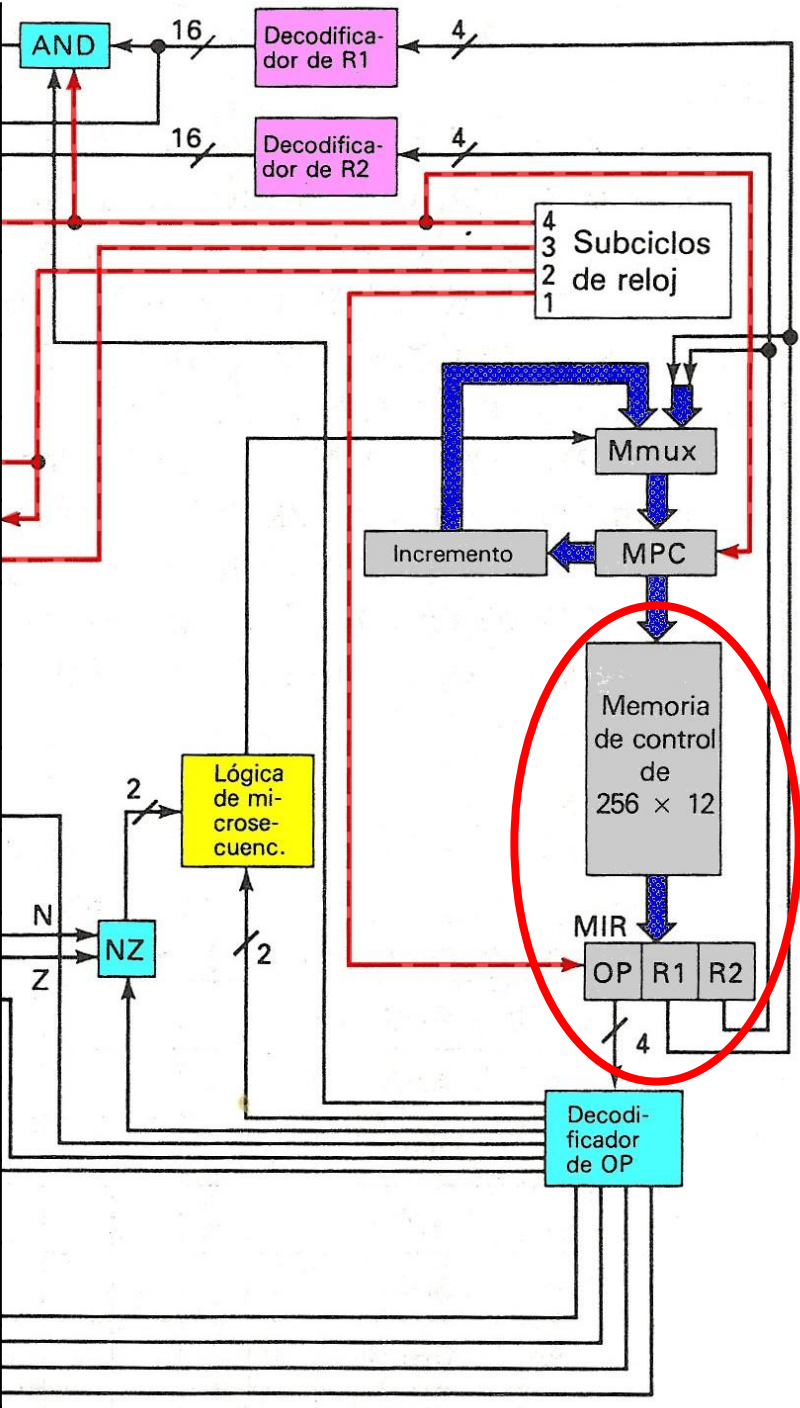
Arquitectura Horizontal



Arquitectura Vertical



OP	Mnemónico	Instrucción
0000	ADD	Suma
0001	AND	Y bit a bit
0010	MOVE	Mueve registro
0011	COMPL	Complemento
0100	LSHIFT	Desplaza a la Izquierda
0101	RSHIFT	Desplaza a la Derecha
0110	GETMBR	Almacena MBR en registro
0111	TEST	Examina registro
1000	BEGRD	Comienza la Lectura
1001	BEGWR	Comienza la Escritura
1010	CONRD	Continúa la Lectura
1011	CONWR	Continúa la Escritura
1100		
1101	NJUMP	Salta si N=true
1110	ZJUMP	Salta si Z=true
1111	UJUMP	Salta siempre



Binario	Mnemónico	Instrucción	Significado
0000	ADD	Suma	r1:=r1+r2
0001	AND	Y bit a bit	r1:=r1&r2
0010	MOVE	Mueve registro	r1:=r2
0011	COMPL	Complemento	r1:=inv(r2)
0100	LSHIFT	Desplaza a la Izquierda	r1:=r2<<1
0101	RSHIFT	Desplaza a la Derecha	r1:=r2>>1
0110	GETMBR	Almacena MBR en registro	r1:=MBR
0111	TEST	Examina registro	N:=(r2<0); Z:=(r2==0)
1000	BEGRD	Comienza la Lectura	MAR:=r1; rd
1001	BEGWR	Comienza la Escritura	MAR:=r1; MBR:=r2; wr
1010	CONRD	Continúa la Lectura	rd
1011	CONWR	Continúa la Escritura	wr
1100			
1101	NJUMP	Salta si N=true	if N then goto r1<<4 r2
1110	ZJUMP	Salta si Z=true	if Z then goto r1<<4 r2
1111	UJUMP	Salta siempre	goto r1<<4 r2

Binario		Mnemónico	Instrucción										Significado									
		Código de operación de microinstrucción		ALUL		SHL		AMUX		MAR		RD		MSLH								
				ALUH		SHH		NZ		AND		MBR		WR		MSLL						
0		ADD						+		+												
1		AND		+				+		+												
2		MOVE	+					+		+												
3		COMPL	+	+				+		+												
4		LSHIFT	+		+			+		+												
5		RSHIFT	+			+		+		+												
6		GETMBR	+					+	+	+												
7		TEST	+																			
8		BEGRD	+									+										
9		BEGWR	+									+		+								
10		CONRD	+									+										
11		CONWR	+											+								
12																						
13		NJUMP	+													+						
14		ZJUMP	+												+							
15		UJUMP	+												+	+						

Deja pasar el dato sin modificaciones

=0)

2; wr

1101	NJUMP	Salta si N=true	if N then goto r1<<4 r2
1110	ZJUMP	Salta si Z=true	if Z then goto r1<<4 r2
1111	UJUMP	Salta siempre	goto r1<<4 r2

Deja pasar el dato sin modificaciones

=0)

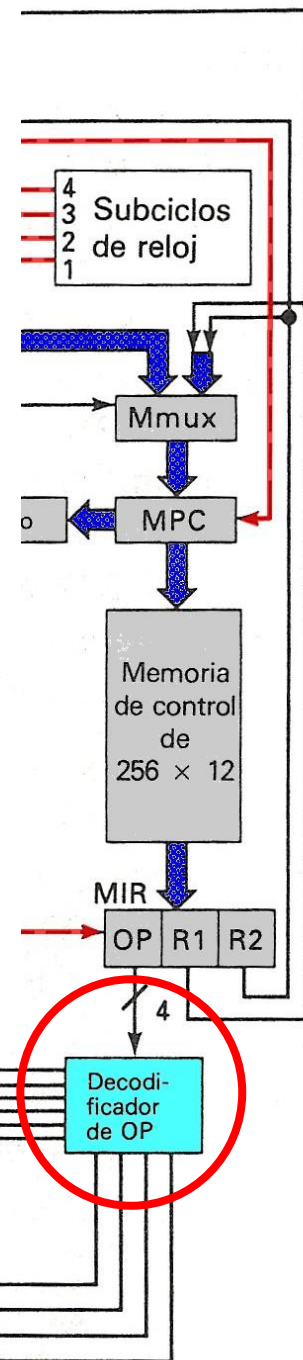
2; wr

Código de operación de microinstrucción

		ALUL	SHL	AMUX	MAR	RD	MSLH	
		ALUH	SHH	NZ	AND	MBR	WR	MSLL

0	ADD				+	+		
1	AND		+		+	+		
2	MOVE	+			+	+		
3	COMPL	+	+		+	+		
4	LSHIFT	+		+		+		
5	RSHIFT	+		+		+		
6	GETMBR	+			+	+	+	
7	TEST	+			+			
8	BEGRD	+				+	+	
9	BEGWR	+				+	+	+
10	CONRD	+				+		
11	CONWR	+					+	
12								
13	NJUMP	+						+
14	ZJUMP						+	
15	U						+	+

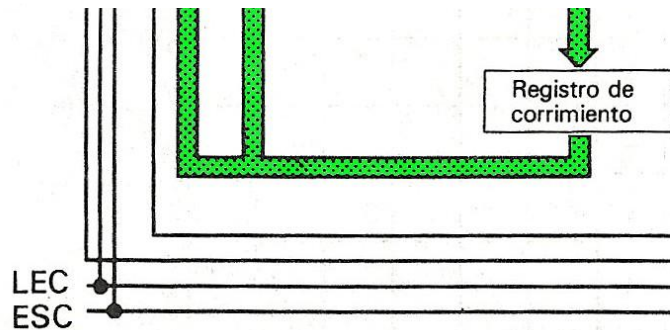
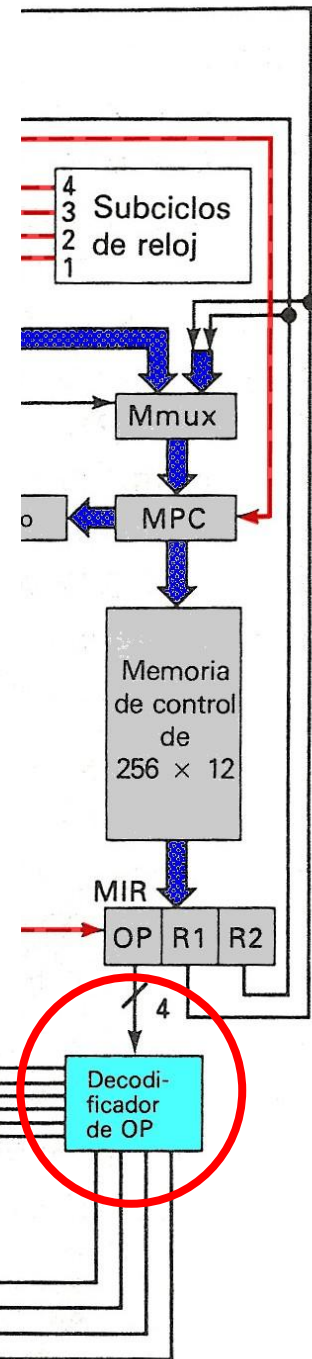
¿Cómo se implementa el Decodificador de Operaciones?



LEC
ESC

Código de operación de microinstrucción

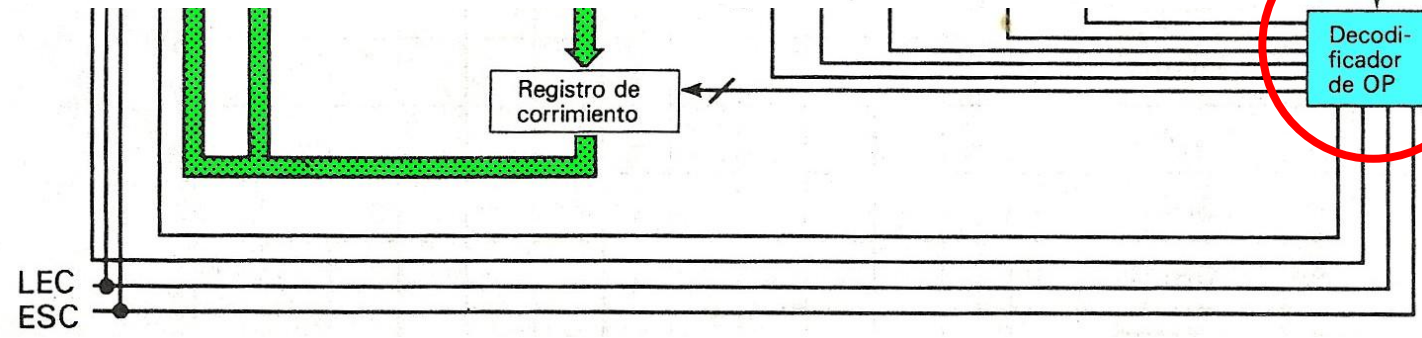
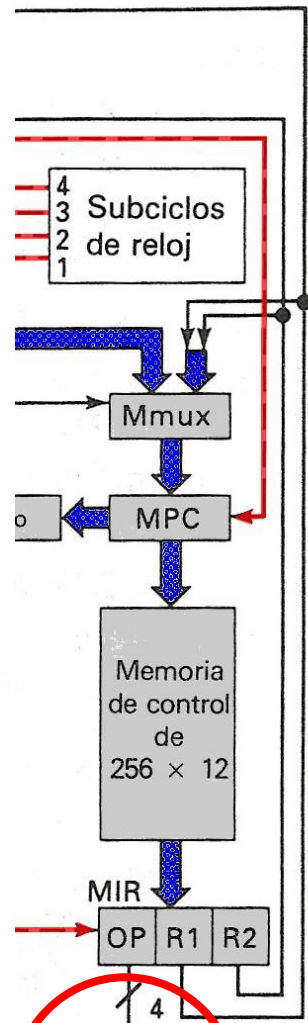
		ALUL	SHL	AMUX	MAR	RD	MSLH	
		ALUH	SHH	NZ	AND	MBR	WR	MSLL
0	ADD			+	+			
1	AND		+	+	+			
2	MOVE	+		+	+			
3	COMPL	+	+	+	+			
4	LSHIFT	+	+		+			
5	RSHIFT	+		+	+			
6	GETMBR	+		+	+	+		
7	TEST	+		+				
8	BEGRD	+				+	+	
9	BEGWR	+				+	+	+
10	CONRD	+				+		
11	CONWR	+					+	
12								
13	NJUMP	+						+
14	ZJUMP	+					+	
15	UJUMP	+					+	+



Código de operación de microinstrucción

		ALUL	SHL	AMUX	MAR	RD	MSLH	
		ALUH	SHH	NZ	AND	MBR	WR	MSLL
0	ADD			+	+			
1	AND		+	+	+			
2	MOVE	+		+	+			
3	COMPL	+	+	+	+			
4	LSHIFT	+	+		+			
5	RSHIFT	+		+				
6	GETMBR	+						
7	TEST	+						
8	BEGRD	+				+		
9	BEGWR	+				+	+	
10	CONRD	+				+		
11	CONWR	+					+	
12								
13	NJUMP	+						+
14	ZJUMP	+					+	
15	UJUMP	+					+	+

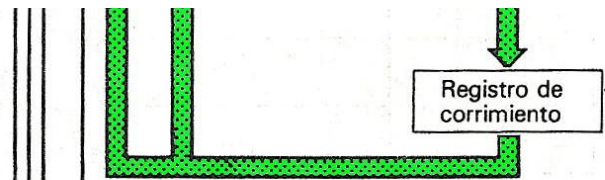
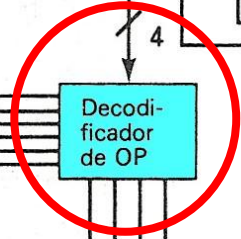
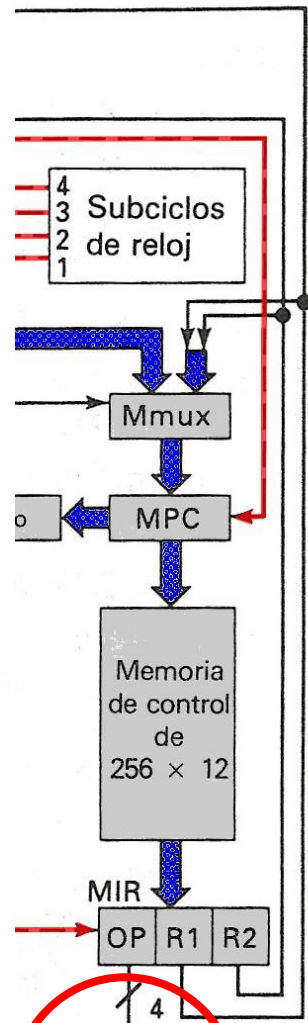
0100



Código de operación de microinstrucción

		ALUH	SHH	NZ	AND	MAR	RD	MSLH	MSLL
0	ADD			+	+				
1	AND		+	+	+				
2	MOVE	+		+	+				
3	COMPL	+	+	+	+				
4	LSHIFT	+	+		+				
5	RSHIFT	+			+				
6	GETMBR	+			+				
7	TEST	+							
8	BEGRD	+				+	+		
9	BEGWR	+				+	+	+	
10	CONRD	+					+		
11	CONWR	+						+	
12									
13	NJUMP	+							+
14	ZJUMP	+						+	
15	UJUMP	+						+	+

0001
0011
=====
00?1

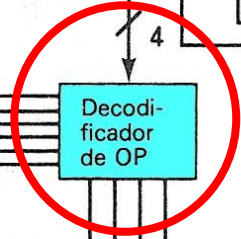
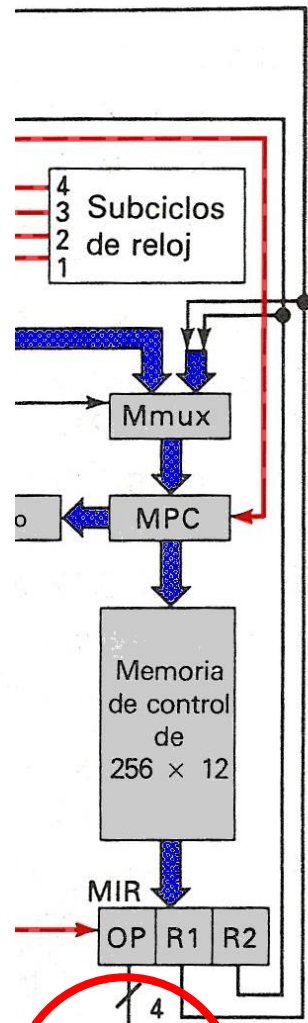


LEC
ESC

Código de operación de microinstrucción

		ALUL	SHL	AMUX	MAR	RD	MSLH	
		ALUH	SHH	NZ	AND	MBR	WR	MSLL
0	ADD			+	+			
1	AND		+	+	+			
2	MOVE	+		+	+			
3	COMPL	+	+		+			
4	LSHIFT	+	+		+			
5	RSHIFT	+			+			
6	GETMBR	+			+			
7	TEST	+						
8	BEGRD	+			+	+		
9	BEGWR	+			+	+	+	
10	CONRD	+				+		
11	CONWR	+					+	
12								
13	NJUMP	+						+
14	ZJUMP	+					+	
15	UJUMP	+					+	+

1000
1001
=====
100?



LEC
ESC

Código de operación
de microinstrucción

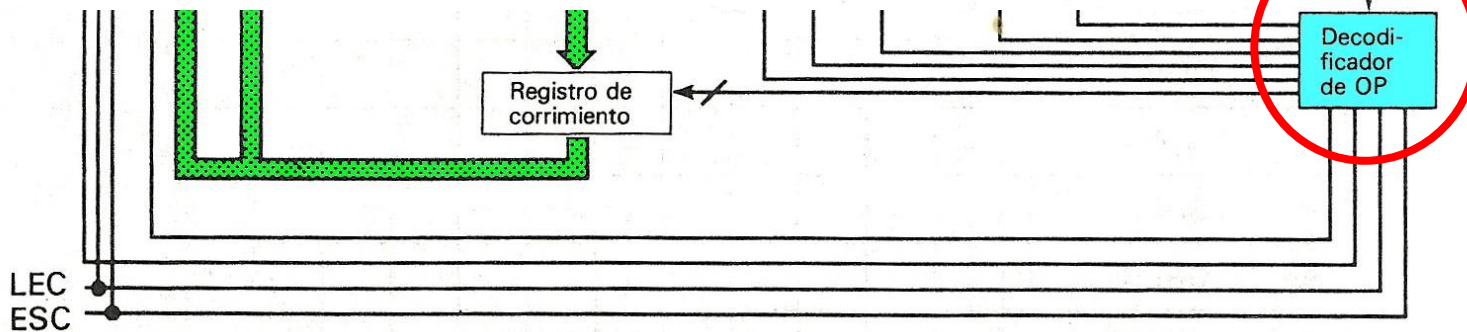
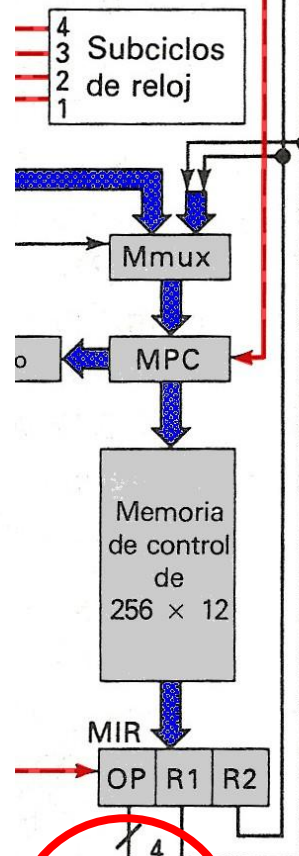
		ALUL	SHL	AMUX	MAR	RD	MSLH	MSLL
		ALUH	SHH	NZ	AND	MBR	WR	
0	ADD			+	+			
1	AND	+		+	+			
2	MOVE	+		+	+			
3	COMPL	+	+	+	+			
4	LSHIFT	+	+		+			
5	RSHIFT	+		+	+			
6	GETMBR	+		+	+			
7	TEST	+		+				
8	BEGRD	+				+	+	
9	BEGWR	+				+	+	+
10	CONRD	+				+		
11	CONWR	+					+	
12								
13	NJUMP	+						+
14	ZJUMP	+					+	
15	UJUMP	+					+	+

1110

1111

=====

111?



0: mar := pc; rd;	23: alu := ac; {IZER}	38: a := ir; {SUBL}	58: a := mbr;
1: rd;	if z then goto 22;	a := a + sp;	mar := ac; mbr := a; wr;
pc := pc + 1;	24: goto 0;	39: mar := a; rd;	wr;
2: ir := mbr;	25: alu := tir;	rd;	goto 0;
tir := lshift(ir);	if n then goto 27;	goto 99;	59: alu := tir;
if n then goto 28;	26: pc:= ir; {JUMP}	40: tir := lshift (tir);	if n then goto 62;
3: tir := lshift(tir);	pc:= band(pc, amask);	if n then goto 46;	60: sp := sp + (-1); {PUSH}
if n then goto 19;	goto 0;	41: alu:= tir;	61: mar:= sp; mbr :=ac; wr;
4: tir := lshift(tir);	27: ac := ir; {LOCO}	if n then goto 44;	wr;
if n then goto 11;	ac :=band(ac, amask);	42: alu:= ac; {JNEG}	goto 0;
5: alu := tir;	goto 0;	if n then goto 22;	62: mar := sp; rd; {POP}
if n then goto 09;	28: tir := lshift (tir):	43: goto 0;	63: rd;
6: mar := ir; rd; {LODD}	if n then goto 40;	44: alu := ac; {JNZE}	sp := sp + 1;
7: rd;	29: tir:= lshift (tir);	if z then goto 0;	64: ac := mbr;
8: ac := mbr;	if n then goto 35;	45: pc := ir;	goto 0;
goto 0;	30: alu := tir;	pc := band (pc, amask);	65: tir := lshift (tir):
9: mar := ir; mbr:= ac; wr;	if n then goto 33;	goto 0;	if n then goto 73;
{STOD}	31: a := ir; {LODL}	46: tir := lshift(tir);	66: alu := tir;
10: wr	a := a + sp;	if n then goto 50;	if n then goto 70;
goto 0;	32: mar := a; rd;	47: sp := sp + (-1); {CALL}	67: mar := sp; rd; {RETN}
11: alu := tir;	rd;	48: mar := sp; mbr:= pc; wr;	68: rd;
if n then goto 15;	ac := mbr;	wr;	sp := sp + 1;
12: mar := ir; rd; {ADD}	goto 0;	49: pc := ir;	69: pc := mbr;
13: rd;	33: a := ir; {STOL}	pc := band(pc, amask);	goto 0;
14: a := mbr;	a := a + sp;	goto 0;	70: a := ac; {SWAP}
ac := ac + a;	34: mar := a; mbr := ac, wr;	50: tir := lshift (tir);	ac := sp;
goto 0;	wr;	if n then goto 55;	sp:= a;
15: mar := ir; rd; {SUBD}	goto 0;	tir := lshift (tir);	goto 0;
16: rd;	35: alu := tir;	if n then goto 52;	71: a := mbr;
99: ac := ac + 1;	if n then goto 38;	52: a := mbr;	goto 76;
17: a := mbr;	36: a := ir; {ADDL}	if n then goto 57;	72: mar := a; wr;
a := inv(a);	a := a + sp;	57: rd;	sp := a; mbr := a;
18: ac := ac + a;	37: mar := a; rd;	rd;	goto 0;
goto 0;	rd;	a := mbr;	73: a := mbr;
19: tir := lshift(tir);	ac := ac + a;	goto 0;	74: a := a + mbr;
if n then goto 25;	goto 0;	54: mar := sp; rd; {POPI}	sp := sp + a;
20: alu := tir;	21: alu := ac; {JPOS}	55: rd;	goto 0;
if n then goto 23;	if n then goto 0;	sp := sp + 1;	
21: alu := ac;	22: pc := ir;		
if n then goto 0;	pc band(pc, amask);		
22: pc := ir;	goto 0;		

Micro-Programa Vertical

0: mar := pc; rd;	23: alu := ac; {TZER}	38: a := ir; {SUBL}	58: a := mbr;
1: rd;			mar := ac; mbr := a; wr;
pc := pc + 1;			wr;
2: ir := mbr;			goto 0;
tir := lshift(ir);	0: mar := pc; rd;	{ciclo principal}	alu := tir;
if n then goto 28;	1: pc := pc + 1; rd;	{incrementa pc}	if n then goto 62;
3: tir := lshift(tir);	2: ir := mbr; if n then goto 28;	{salva y decodifica mbr}	sp := sp + (-1); {PUSH}
if n then goto 19;	3: tir := lshift(ir + ir); if n then goto 19;		mar := sp; mbr := ac; wr;
4: tir := lshift(tir);	4: tir := lshift(tir); if n then goto 11;	{000x o 001x?}	wr;
if n then goto 0;	5: alu := tir; if n then goto 9;	{0000 o 0001?}	goto 0;
5: alu := tir;	6: mar := ir; rd;	{0000 = LODD}	mar := sp; rd; {POP}
if n then goto 0;	7: rd;		rd;
6: mar := ir;	8: ac := mbr; goto 0;		sp := sp + 1;
7: rd;			64: ac := mbr;
8: a := mbr; {STOD}			goto 0;
9: wr := mbr; goto 0;	29: tir := lshift(tir);	if z then goto 0;	65: tir := lshift(tir);
10: wr := mbr; goto 0;	if n then goto 35;		if n then goto 73;
11: alu := tir;	30: alu := tir;	45: pc := ir;	66: alu := tir;
if n then goto 15;	if n then goto 33;	pc := band(pc, amask);	if n then goto 70;
12: mar := ir; rd; {ADD}	31: a := ir; {LODL}	goto 0;	67: mar := sp; rd; {RETN}
13: rd;	a := a + sp;	46: tir := lshift(tir);	68: rd;
14: a := mbr;	32: mar := a; rd;	if n then goto 50;	sp := sp + 1;
ac := ac + a;	rd;	47: sp := sp + (-1); {CALL}	69: pc := mbr;
goto 0;	ac := mbr;	48: mar := sp; mbr := pc; wr;	goto 0;
15: mar := ir; rd; {SUBD}	goto 0;	49: pc := ir;	70: a := ac; {SWAP}
16: rd;	33: a := ir; {STOL}	pc := band(pc, amask);	ac := sp;
99: ac := ac + 1;	a := a + sp;	goto 0;	sp := a;
17: a := mbr;	34: mar := a; mbr := ac, wr;	50: tir := lshift(tir);	goto 0;
a := inv(a);	wr;	if n then goto 55;	
18: ac := ac + a;	goto 0;	tir := lshift(tir);	
goto 0;	35: alu := tir;	if n then goto 58;	
19: tir := lshift(tir);	if n then goto 38;	52: a := mbr; {TNSP}	
if n then goto 25;	36: a := ir; {ADDL}	a := a + sp;	
20: alu := tir;	a := a + sp;	54: mar := mbr; {POPI}	
if n then goto 23;	37: mar := a; rd;	wr;	
21: alu := ac; {JPOS}	rd;	goto 0;	
if n then goto 0;	a := mbr;	56: mar := sp; rd; {POPI}	
22: pc := ir;	ac := ac + a;	57: rd;	
pc band(pc, amask);	goto 0;	sp := sp + 1;	
goto 0;			

Micro-Programa Horizontal

Micro-Programa Vertical

0: mar := pc; rd;

```
1: rd;
  pc := pc + 1;
2: ir := mbr;
  tir := lshift(ir);
  if n then goto 28;
3: tir := lshift(tir);
  if n then goto 19;
4: tir := lshift(tir);
  if n then goto 11;
5: alu := tir;
  if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
  goto 0;
9: mar := ir; mbr:= ac; wr;
  {STOD}
10: wr
  goto 0;
11: alu := tir;
  if n then goto 15;
12: mar := ir; rd; {ADD}
13: rd;
14: a := mbr;
  ac := ac + a;
  goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
  a := inv(a);
18: ac := ac + a;
  goto 0;
19: tir := lshift(tir);
  if n then goto 25;
20: alu := tir;
  if n then goto 23;
21: alu := ac; {JPOS}
  if n then goto 0;
22: pc := ir;
  pc band(pc, amask);
  goto 0;
```

23: alu := ac; {TZER}

```
0: mar := pc; rd;
1: pc := pc + 1; rd;
2: ir := mbr; if n then goto 28;
3: tir := lshift(ir + ir); if n then goto 19;
4: tir := lshift(tir); if n then goto 11;
5: alu := tir; if n then goto 9;
6: mar := ir; rd;
7: rd;
8: ac := mbr; goto 0;
```

```
29: tir:= lshift (tir);
  if n then goto 35;
30: alu := tir;
  if n then goto 33;
31: a := ir; {LODL}
  a := a + sp;
32: mar := a; rd;
  rd;
  ac := mbr;
  goto 0;
33: a := ir; {STOL}
  a := a + sp;
34: mar := a; mbr := ac, wr;
  wr;
  goto 0;
35: alu := tir;
  if n then goto 38;
36: a := ir; {ADDL}
  a := a + sp;
37: mar := a; rd;
  rd;
  a := mbr;
  ac := ac + a;
  goto 0;
```

38: a := ir; {SUBL}

```
{ciclo principal}
{incrementa pe}
{salva y decodifica mbr}
{000x o 001x?}
{0000 o 0001?}
{0000 = LODD}
```

```
if z then goto 0;
45: pc := ir;
  pc := band (pc, amask);
  goto 0;
46: tir := lshift(tir);
  if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
  wr;
49: pc := ir;
  pc := band(pc, amask);
  goto 0;
50: tir := lshift (tir );
  if n then goto 55;
  tir := lshift (tir );
  if n then goto 52;
52: a := mbr;
  a := a + sp;
  goto 0;
54: mar := sp; rd;
  wr;
  goto 0;
56: mar := sp; rd; {POPI}
57: rd;
  sp := sp + 1;
```

```
58: a := mbr;
  mar := ac; mbr := a; wr;
  wr;
  goto 0;
  alu := tir;
  if n then goto 62;
  sp := sp + (-1); {PUSH}
  mar:= sp; mbr :=ac; wr;
  wr;
  goto 0;
  mar := sp; rd; {POP}
  rd;
  sp := sp + 1;
64: ac := mbr;
  goto 0;
65: tir := lshift (tir );
  if n then goto 73;
66: alu := tir;
  if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
  sp := sp + 1;
69: pc := mbr;
  goto 0;
70: a := ac; {SWAP}
  ac := sp;
  sp:= a;
  goto 0;
```

```
;
  if n then goto 76;
  {TNSP}
  a := mbr;
  a := a + sp;
  sp := sp + a;
  goto 0;
```

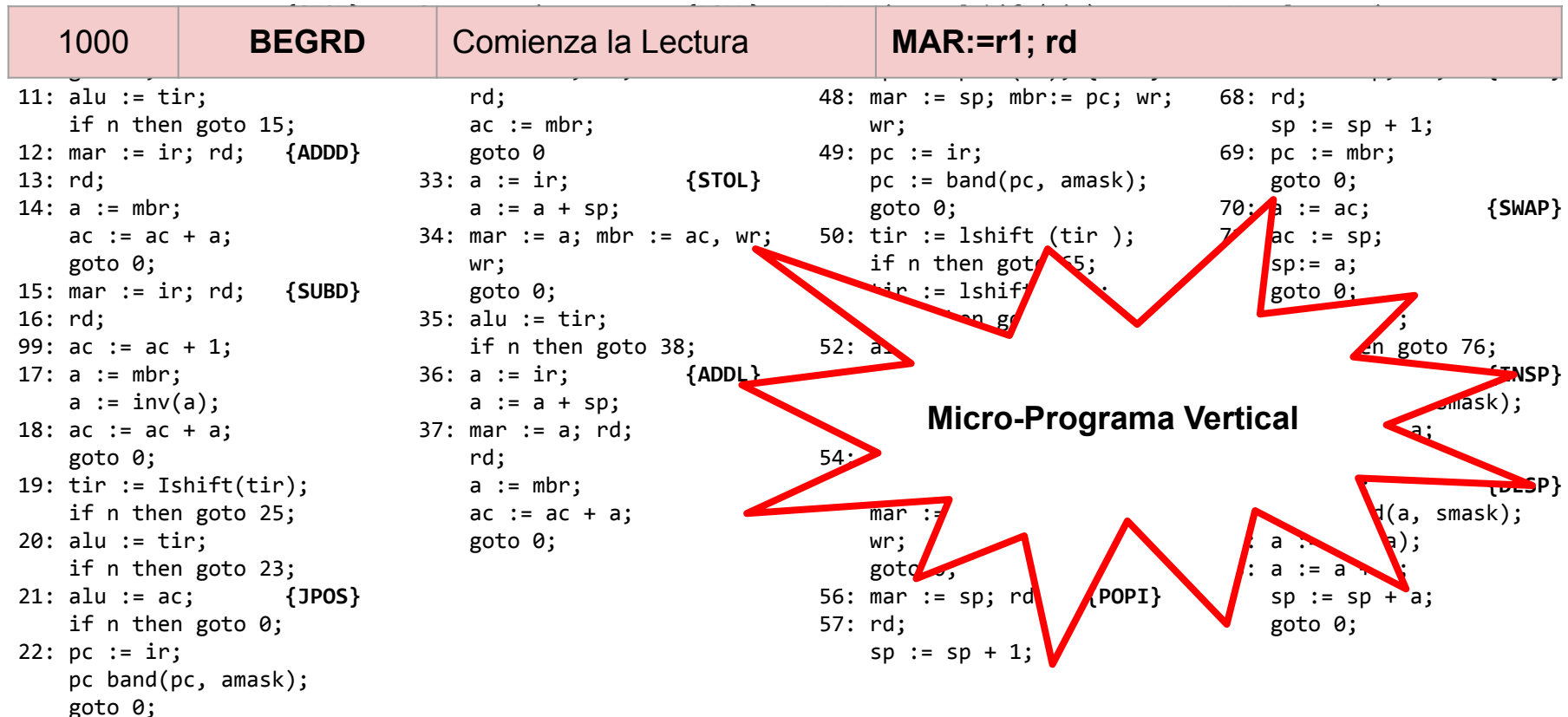
Micro-Programa Vertical

```

58:  a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
    alu := tir;
    if n then goto 62;
    sp := sp + (-1); {PUSH}
    mar:= sp; mbr :=ac; wr;
    wr;
    goto 0;
    mar := sp; rd;    {POP}
    rd;
    sp := sp + 1;
64:  ac := mbr;
    goto 0;
65:  tir := lshift (tir );
    if n then goto 73;

```

0: <i>mar</i> := <i>pc</i> ; <i>rd</i> ;	{ciclo principal}	
1: <i>pc</i> := <i>pc</i> + 1; <i>rd</i> ;	{incrementa <i>pc</i> }	
2: <i>ir</i> := <i>mbr</i> ; if <i>n</i> then goto 28;	{salva y decodifica <i>mbr</i> }	
3: <i>tir</i> := <i>lshift</i> (<i>ir</i> + <i>ir</i>); if <i>n</i> then goto 19;		
4: <i>tir</i> := <i>lshift</i> (<i>tir</i>); if <i>n</i> then goto 11;	{000x o 001x?}	
5: <i>alu</i> := <i>tir</i> ; if <i>n</i> then goto 9;	{0000 o 0001?}	
6: <i>mar</i> := <i>ir</i> ; <i>rd</i> ;	{0000 = LODD}	
7: <i>rd</i> ;		
8: <i>ac</i> := <i>mbr</i> ; goto 0;		
29: <i>tir</i> := <i>lshift</i> (<i>tir</i>);	if <i>z</i> then goto 0;	64
if <i>n</i> then goto 35;	45: <i>pc</i> := <i>ir</i> ;	
30: <i>alu</i> := <i>tir</i> ;	<i>pc</i> := <i>band</i> (<i>pc</i> , <i>amask</i>);	65
if <i>n</i> then goto 33;	goto 0;	



```

0: mar := pc; rd;
1: rd;
  pc := pc + 1;
2: ir := mbr;
  tir := lshift(ir);
  if n then goto 28;
3: tir := lshift(tir);
  if n then goto 19;
4: tir := lshift(tir);
  if n then goto 11;
5: alu := tir;
  if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
  goto 0;
9: mar := ir; mbr:= ac; wr;
  {STOD}
10: wr
  goto 0;
11: alu := tir;
  if n then goto 15;
12: mar := ir; rd; {ADD}
13: rd;
14: a := mbr;
  ac := ac + a;
  goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
  a := inv(a);
18: ac := ac + a;
  goto 0;
19: tir := lshift(tir);
  if n then goto 25;
20: alu := tir;
  if n then goto 23;
21: alu := ac; {JPOS}
  if n then goto 0;
22: pc := ir;
  pc band(pc, amask);
  goto 0;

```

```

23: alu := ac; {TZER}
38: a := ir; {SUBL}
58: a := mbr;
  mar := ac; mbr := a; wr;
  wr;
  goto 0;
  alu := tir;
  if n then goto 62;
  sp := sp + (-1); {PUSH}
  mar:= sp; mbr:=ac; wr;
  wr;
  goto 0;
  mar := sp; rd; {POP}
  rd;
  sp := sp + 1;
64: ac := mbr;
  goto 0;
65: tir := lshift (tir );
  if n then goto 73;
66: alu := tir;
  if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
  sp := sp + 1;
69: pc := mbr;
  goto 0;
70: a := ac; {SWAP}
  ac := sp;
  sp:= a;
  goto 0;
  ;
  en goto 76;
  mask);
  a;
  {JESP}
  d(a, smask);
  a := a;
  a := a;
  sp := sp + a;
  goto 0;
54:
  mar :=
  wr;
  goto 0;
56: mar := sp; rd; {POPI}
57: rd;
  sp := sp + 1;

```

Micro-Programa Vertical


```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;

```

```

23: alu := ac; {TZER}
38: a := ir; {SUBL}
58: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
   alu := tir;
   if n then goto 62;
   sp := sp + (-1); {PUSH}
   mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
   mar := sp; rd; {POP}
   rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;

```

```

0: mar := pc; rd; {ciclo principal}
1: pc := pc + 1; rd; {incrementa pc}
2: ir := mbr; if n then goto 28; {salva y decodifica mbr}
3: tir := lshift (ir + ir ); if n then goto 19;
4: tir := lshift (tir ); if n then goto 11; {000x o 001x?}
5: alu := tir; if n then goto 9; {0000 o 0001?}
6: mar := ir; rd; {0000 = LODD}
7: rd;
8: ac := mbr; goto 0;
29: tir:= lshift (tir);
   if n then goto 35;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;

```

1010	CONRD	Continúa la Lectura	rd
------	-------	---------------------	----

```

11: alu := tir;
   if n then goto 15;
12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;

```

```

rd;
ac := mbr;
goto 0
33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;

```

```

48: mar := sp; mbr:= pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 55;
   tir := lshift (tir );
   if n then goto 60;
52: a := mbr;
   ac := ac + a;
   goto 0;
54: mar := sp; rd; {POPI}
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;

```

```

68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
   ac := sp;
   sp:= a;
   goto 0;
71: a := mbr;
   if n then goto 76;
   a := inv(a);
   a := a + sp;
   goto 0;
72: a := mbr;
   a := a + sp;
   goto 0;
73: a := mbr;
   a := a + sp;
   goto 0;
74: a := mbr;
   a := a + sp;
   goto 0;
75: a := mbr;
   a := a + sp;
   goto 0;
76: a := mbr;
   a := a + sp;
   goto 0;
77: a := mbr;
   a := a + sp;
   goto 0;
78: a := mbr;
   a := a + sp;
   goto 0;
79: a := mbr;
   a := a + sp;
   goto 0;
80: a := mbr;
   a := a + sp;
   goto 0;
81: a := mbr;
   a := a + sp;
   goto 0;
82: a := mbr;
   a := a + sp;
   goto 0;
83: a := mbr;
   a := a + sp;
   goto 0;
84: a := mbr;
   a := a + sp;
   goto 0;
85: a := mbr;
   a := a + sp;
   goto 0;
86: a := mbr;
   a := a + sp;
   goto 0;
87: a := mbr;
   a := a + sp;
   goto 0;
88: a := mbr;
   a := a + sp;
   goto 0;
89: a := mbr;
   a := a + sp;
   goto 0;
90: a := mbr;
   a := a + sp;
   goto 0;
91: a := mbr;
   a := a + sp;
   goto 0;
92: a := mbr;
   a := a + sp;
   goto 0;
93: a := mbr;
   a := a + sp;
   goto 0;
94: a := mbr;
   a := a + sp;
   goto 0;
95: a := mbr;
   a := a + sp;
   goto 0;
96: a := mbr;
   a := a + sp;
   goto 0;
97: a := mbr;
   a := a + sp;
   goto 0;
98: a := mbr;
   a := a + sp;
   goto 0;
99: a := mbr;
   a := a + sp;
   goto 0;

```

Micro-Programa Vertical

0: mar := pc; rd;	23: alu := ac; {TZER}	38: a := ir; {SUBL}	58: a := mbr;
1: rd;	0: mar := pc; rd; {ciclo principal}		mar := ac; mbr := a; wr;
pc := pc + 1;	1: pc := pc + 1; rd; {incrementa pc}		wr;
2: ir := mbr;	2: ir := mbr; if n then goto 28; {salva y decodifica mbr}		goto 0;
tir := lshift(ir);	3: tir := lshift(ir + ir); if n then goto 19;		alu := tir;
if n then goto 28;	4: tir := lshift(tir); if n then goto 11; {000x o 001x?}		if n then goto 62;
3: tir := lshift(tir);	5: alu := tir; if n then goto 9; {0000 o 0001?}		sp := sp + (-1); {PUSH}
if n then goto 19;	6: mar := ir; rd; {0000 = LODD}		mar := sp; mbr := ac; wr;
4: tir := lshift(tir);	7: rd;		wr;
if n then goto 11;	8: ac := mbr; goto 0;		goto 0;
5: alu := tir;			mar := sp; rd; {POP}
if n then goto 09;			rd;
6: mar := ir; rd; {LODD}			sp := sp + 1;
7: rd;			64: ac := mbr;
8: ac := mbr;			goto 0;
goto 0;			65: tir := lshift(tir);
9: mar := ir; mbr := ac; wr;			if n then goto 73;

1010	CONRD	Continúa la Lectura	rd
0000	ADD	Suma	r1:=r1+r2

12: mar := ir; rd; {ADDD}	goto 0	49: pc := ir;	69: pc := mbr;
13: rd;	33: a := ir; {STOL}	pc := band(pc, amask);	goto 0;
14: a := mbr;	a := a + sp;	goto 0;	70: a := ac; {SWAP}
ac := ac + a;	34: mar := a; mbr := ac, wr;	50: tir := lshift(tir);	ac := sp;
goto 0;	wr;	if n then goto 55;	sp := a;
15: mar := ir; rd; {SUBD}	goto 0;	tir := lshift(tir);	goto 0;
16: rd;	35: alu := tir;		
99: ac := ac + 1;	if n then goto 38;	52: a := ir;	en goto 76;
17: a := mbr;	{ADDL}	a := a + sp;	{TNSP}
a := inv(a);	36: a := ir;	a := a + sp;	amask);
18: ac := ac + a;	37: mar := a; rd;	rd;	a;
goto 0;	rd;	a := mbr;	{LSP}
19: tir := lshift(tir);	ac := ac + a;	goto 0;	id(a, smask);
if n then goto 25;	goto 0;	mar :=	a := a;
20: alu := tir;		wr;	a := a;
if n then goto 23;		goto 0;	sp := sp + a;
21: alu := ac; {JPOS}		56: mar := sp; rd; {POPI}	goto 0;
if n then goto 0;		57: rd;	
22: pc := ir;		sp := sp + 1;	
pc band(pc, amask);			
goto 0;			

Micro-Programa Vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr
   goto 0;
11: alu := tir;
   if n then goto 15;
12: mar := ir; rd; {ADD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;

```

```

23: alu := ac; {TZER}
38: a := ir; {SUBL}
58: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
   alu := tir;
   if n then goto 62;
   sp := sp + (-1); {PUSH}
   mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
   mar := sp; rd; {POP}
   rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;
66: alu := tir;
   if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
   ac := sp;
   sp:= a;
   goto 0;
   ;
   en goto 76;
   {TNSP}
   amask);
   a;
   {LSP}
   d(a, smask);
   a := a;
   a := a;
   sp := sp + a;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;

```

```

29: tir:= lshift (tir);
   if n then goto 35;
30: alu := tir;
   if n then goto 33;
31: a := ir; {LODL}
   a := a + sp;
32: mar := a; rd;
   rd;
   ac := mbr;
   goto 0;
33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;
46: tir := lshift(tir);
   if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 55;
   tir := lshift (tir );
   if n then goto 52;
52: a := mbr;
   ac := ac + a;
   goto 0;
54: mar := sp; rd; {POPI}
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;

```

Micro-Programa Vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;

```

```

23: alu := ac; {TZER}
38: a := ir; {SUBL}
58: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
   alu := tir;
   if n then goto 62;
   sp := sp + (-1); {PUSH}
   mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
   mar := sp; rd; {POP}
   rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;

```

```

0: mar := pc; rd; {ciclo principal}
1: pc := pc + 1; rd; {incrementa pc}
2: ir := mbr; if n then goto 28; {salva y decodifica mbr}
3: tir := lshift(ir + ir); if n then goto 19;
4: tir := lshift(tir); if n then goto 11; {000x o 001x?}
5: alu := tir; if n then goto 9; {0000 o 0001?}
6: mar := ir; rd; {0000 = LODD}
7: rd;
8: ac := mbr; goto 0;
29: tir:= lshift (tir);
   if n then goto 35;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;

```

0110	GETMBR	Almacena MBR en registro	r1:=MBR
------	--------	--------------------------	---------

```

11: alu := tir;
   if n then goto 15;
12: mar := ir; rd; {ADD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;

```

```

rd;
ac := mbr;
goto 0
33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;

```

```

48: mar := sp; mbr:= pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 55;
   tir := lshift (tir );
   if n then goto 60;
52: a := mbr;
   ac := ac + a;
   goto 0;
54: mar := sp; rd; {POPI}
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;

```

```

68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
   ac := sp;
   sp:= a;
   goto 0;
71: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
72: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
73: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
74: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
75: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
76: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
77: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
78: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
79: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
80: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
81: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
82: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
83: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
84: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
85: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
86: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
87: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
88: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
89: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
90: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
91: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
92: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
93: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
94: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
95: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
96: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
97: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
98: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;
99: a := mbr;
   if n then goto 76;
   ac := a;
   goto 0;

```

Micro-Programa Vertical


```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;

```

```

23: alu := ac; {TZER}
38: a := ir; {SUBL}
58: a := mbr;
   mar := ac; mbr := a; wr;
   goto 0;
   alu := tir;
   if n then goto 62;
   sp := sp + (-1); {PUSH}
   mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
   mar := sp; rd; {POP}
   rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;

```

```

0: mar := pc; rd; {ciclo principal}
1: pc := pc + 1; rd; {incrementa pc}
2: ir := mbr; if n then goto 28; {salva y decodifica mbr}
3: tir := lshift(ir + ir); if n then goto 19;
4: tir := lshift(tir); if n then goto 11; {000x o 001x?}
5: alu := tir; if n then goto 9; {0000 o 0001?}
6: mar := ir; rd; {0000 = LODD}
7: rd;
8: ac := mbr; goto 0;
29: tir:= lshift (tir);
   if n then goto 35;
30: alu := tir;
   if n then goto 33;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;

```

0110	GETMBR	Almacena MBR en registro	r1:=MBR
0100	LSHIFT	Desplaza a la Izquierda	r1:=r2<<1

```

12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
19: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;

```

```

33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;

```

```

49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 55;
   tir := lshift (tir );
   if n then goto 60;
52: a := mbr;
   a := a + sp;
54: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;

```

```

69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
   ac := sp;
   sp:= a;
   goto 0;
71: a := mbr;
   a := a + sp;
   goto 0;
72: a := mbr;
   a := a + sp;
   goto 0;
73: a := mbr;
   a := a + sp;
   goto 0;
74: a := mbr;
   a := a + sp;
   goto 0;
75: a := mbr;
   a := a + sp;
   goto 0;
76: a := mbr;
   a := a + sp;
   goto 0;
77: a := mbr;
   a := a + sp;
   goto 0;
78: a := mbr;
   a := a + sp;
   goto 0;
79: a := mbr;
   a := a + sp;
   goto 0;
80: a := mbr;
   a := a + sp;
   goto 0;
81: a := mbr;
   a := a + sp;
   goto 0;
82: a := mbr;
   a := a + sp;
   goto 0;
83: a := mbr;
   a := a + sp;
   goto 0;
84: a := mbr;
   a := a + sp;
   goto 0;
85: a := mbr;
   a := a + sp;
   goto 0;
86: a := mbr;
   a := a + sp;
   goto 0;
87: a := mbr;
   a := a + sp;
   goto 0;
88: a := mbr;
   a := a + sp;
   goto 0;
89: a := mbr;
   a := a + sp;
   goto 0;
90: a := mbr;
   a := a + sp;
   goto 0;
91: a := mbr;
   a := a + sp;
   goto 0;
92: a := mbr;
   a := a + sp;
   goto 0;
93: a := mbr;
   a := a + sp;
   goto 0;
94: a := mbr;
   a := a + sp;
   goto 0;
95: a := mbr;
   a := a + sp;
   goto 0;
96: a := mbr;
   a := a + sp;
   goto 0;
97: a := mbr;
   a := a + sp;
   goto 0;
98: a := mbr;
   a := a + sp;
   goto 0;
99: a := mbr;
   a := a + sp;
   goto 0;

```

Micro-Programa Vertical

```

0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(ir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;

```

```

23: alu := ac; {TZER}
38: a := ir; {SUBL}
58: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
   alu := tir;
   if n then goto 62;
   sp := sp + (-1); {PUSH}
   mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
   mar := sp; rd; {POP}
   rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;

```

```

0: mar := pc; rd; {ciclo principal}
1: pc := pc + 1; rd; {incrementa pc}
2: ir := mbr; if n then goto 28; {salva y decodifica mbr}
3: tir := lshift(ir + ir); if n then goto 19;
4: tir := lshift(tir); if n then goto 11; {000x o 001x?}
5: alu := tir; if n then goto 9; {0000 o 0001?}
6: mar := ir; rd; {0000 = LODD}
7: rd;
8: ac := mbr; goto 0;
29: tir:= lshift (tir);
   if n then goto 35;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;

```

0110	GETMBR	Almacena MBR en registro	r1:=MBR
0100	LSHIFT	Desplaza a la Izquierda	r1:=r2<<1
1101	NJUMP	Salta si N=true	if N then goto r1<<4 r2

```

ac := ac + a;
goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;

```

```

34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;

```

```

50: tir := lshift (tir );
   if n then goto 55;
   tir := lshift (tir );
   if n then goto 60;
52: a := mbr;
   a := a + sp;
54: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;

```

```

7: ac := sp;
   sp:= a;
   goto 0;
   a := mbr;
   a := a + sp;
   goto 76;
   a := mbr;
   a := a + sp;
   goto 0;

```

Micro-Programa Vertical

Nano-Programación

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

Micro-Instrucciones repetidas

```

0: mar := pc; rd;
1: rd;
  pc := pc + 1;
2: ir := mbr;
  tir := lshift(tir);
  if n then goto 28;
3: tir := lshift(tir);
  if n then goto 19;
4: tir := lshift(tir);
  if n then goto 11;
5: alu := tir;
  if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
  goto 0;
9: mar := ir; mbr:= ac; wr; {STOD}
10: wr
    goto 0;
11: alu := tir;
  if n then goto 15;
12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
  ac := ac + a;
  goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
99: ac := ac + 1;
17: a := mbr;
  a := inv(a);
18: ac := ac + a;
  goto 0;
19: tir := lshift(tir);
  if n then goto 25;
20: alu := tir;
  if n then goto 23;
21: alu := ac; {JPOS}
  if n then goto 0;
22: pc := ir;
  pc band(pc, amask);
  goto 0;

```

```

    if n then goto 27;
26: pc := ir;                {JUMP}
    pc := band(pc, amask);
    goto 0;
27: ac := ir;                {LOCO}
    ac := band(ac, amask);
    goto 0;
28: tir := lshift (tir);
    if n then goto 40;
29: tir := lshift (tir);
    if n then goto 35;
30: alu := tir;
    if n then goto 33;
31: a := ir;                {LODL}
    a := a + sp;
32: mar := a; rd;
    rd;
    ac := mbr;
    goto 0
33: a := ir;                {STOL}
    a := a + sp;
34: mar := a; mbr := ac, wr;
    wr;
    goto 0;
35: alu := tir;
    if n then goto 38;
36: a := ir;                {ADDL}
    a := a + sp;
37: mar := a; rd;
    rd;
    a := mbr;
    ac := ac + a;
    goto 0;

```

```

goto 99;
40: tir := lshift(tir);
    if n then goto 46;
41: alu:= tir;
    if n then goto 44;
42: alu:= ac;           {JNEG}
    if n then goto 22;
43: goto 0;
44: alu := ac;           {JNZE}
    if z then goto 0;
45: pc := ir;
    pc := band (pc, amask);
    goto 0;
46: tir := lshift(tir);
    if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
    wr;
49: pc := ir;
    pc := band(pc, amask);
    goto 0;
50: tir := lshift (tir );
    if n then goto 65;
51: tir := lshift(tir);
    if n then goto 59;
52: alu := tir;
    if n then goto 56;
53: mar := ac; rd;     {PS}
    rd;
54: sp := sp + (-1);
55: a := mbr;
    mar := sp; mbr:=a;
    wr;
    goto 0;
56: mar := sp; rd;     {POPI}
57: rd;
    sp := sp + 1;

```

```

48: a := mbr;
    mar := ac; mbr := a; wr;
    wr;
    goto 0;
59: alu := tir;
    if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar := sp; mbr := ac; wr;
    wr;
    goto 0;
62: mar := sp; rd; {POP}
63: rd;
    sp := sp + 1;
64: ac := mbr;
    goto 0;
65: tir := lshift (tir);
    if n then goto 73;
66: alu := tir;
    if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
    sp := sp + 1;
69: pc := mbr;
    goto 0;
70: a := ac; {SWAP}
71: ac := sp;
72: sp := a;
    goto 0;
73: alu := a;
    if n then goto 76;
    {INSP}
    ask);
    {DESP}
    ask);
    : 1;
    sp := + a;
    goto 0;

```

Micro-Programa Vertical

Nano programación

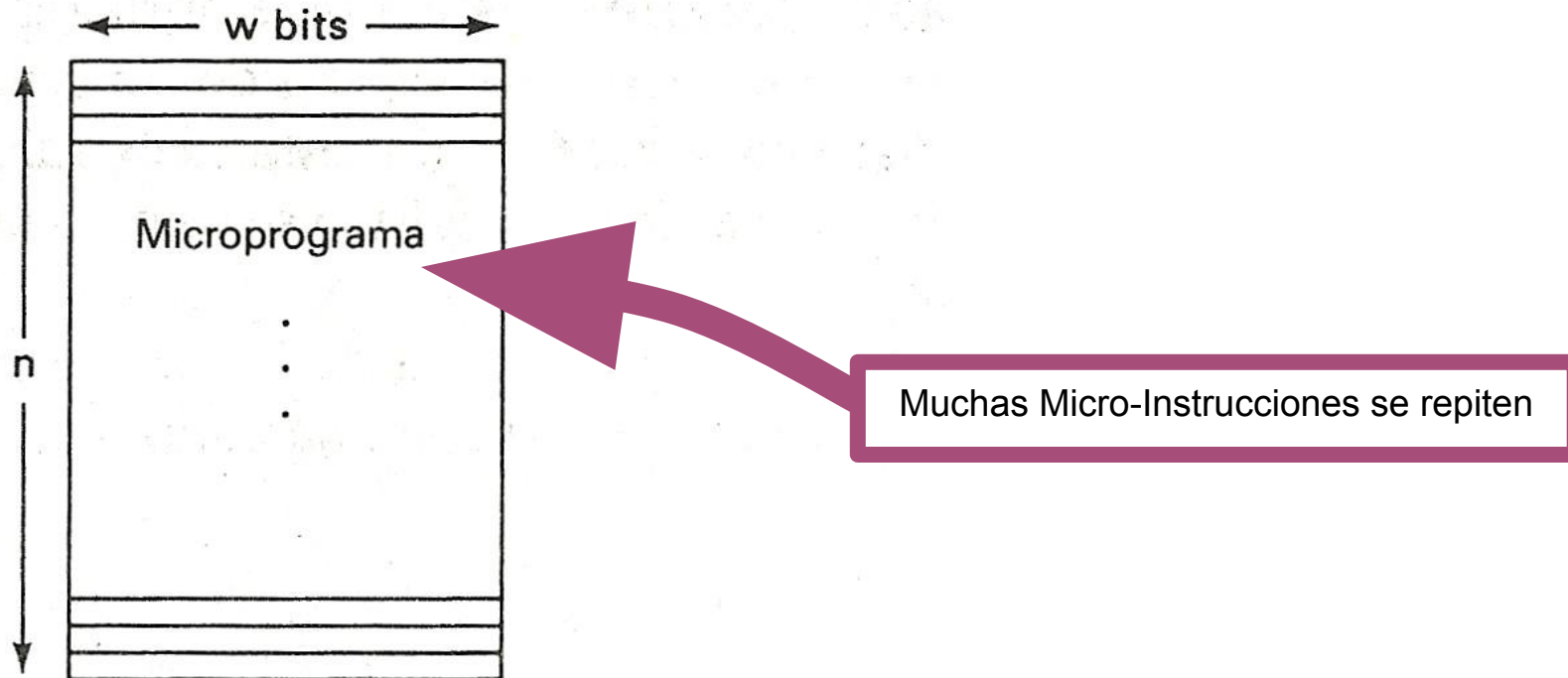
```
0: mar := pc; rd;
1: rd;
   pc := pc + 1;
2: ir := mbr;
   tir := lshift(tir);
   if n then goto 28;
3: tir := lshift(tir);
   if n then goto 19;
4: tir := lshift(tir);
   if n then goto 11;
5: alu := tir;
   if n then goto 09;
6: mar := ir; rd; {LODD}
7: rd;
8: ac := mbr;
   goto 0;
9: mar := ir; mbr:= ac; wr;
   {STOD}
10: wr;
   goto 0;
11: alu := tir;
   if n then goto 15;
12: mar := ir; rd; {ADDD}
13: rd;
14: a := mbr;
   ac := ac + a;
   goto 0;
15: mar := ir; rd; {SUBD}
16: rd;
19: ac := ac + 1;
17: a := mbr;
   a := inv(a);
18: ac := ac + a;
   goto 0;
19: tir := lshift(tir);
   if n then goto 25;
20: alu := tir;
   if n then goto 23;
21: alu := ac; {JPOS}
   if n then goto 0;
22: pc := ir;
   pc band(pc, amask);
   goto 0;
```

```
   if n then goto 27;
26: pc:= ir; {JUMP}
   pc:= band(pc, amask);
   goto 0;
27: ac := ir; {LOCO}
   ac :=band(ac, amask);
   goto 0;
28: tir := lshift (tir);
   if n then goto 40;
29: tir := lshift (tir);
   if n then goto 35;
30: alu := tir;
   if n then goto 33;
31: a := ir; {LODL}
   a := a + sp;
32: mar := a; rd;
   rd;
   ac := mbr;
   goto 0
33: a := ir; {STOL}
   a := a + sp;
34: mar := a; mbr := ac, wr;
   wr;
   goto 0;
35: alu := tir;
   if n then goto 38;
36: a := ir; {ADDL}
   a := a + sp;
37: mar := a; rd;
   rd;
   a := mbr;
   ac := ac + a;
   goto 0;
```

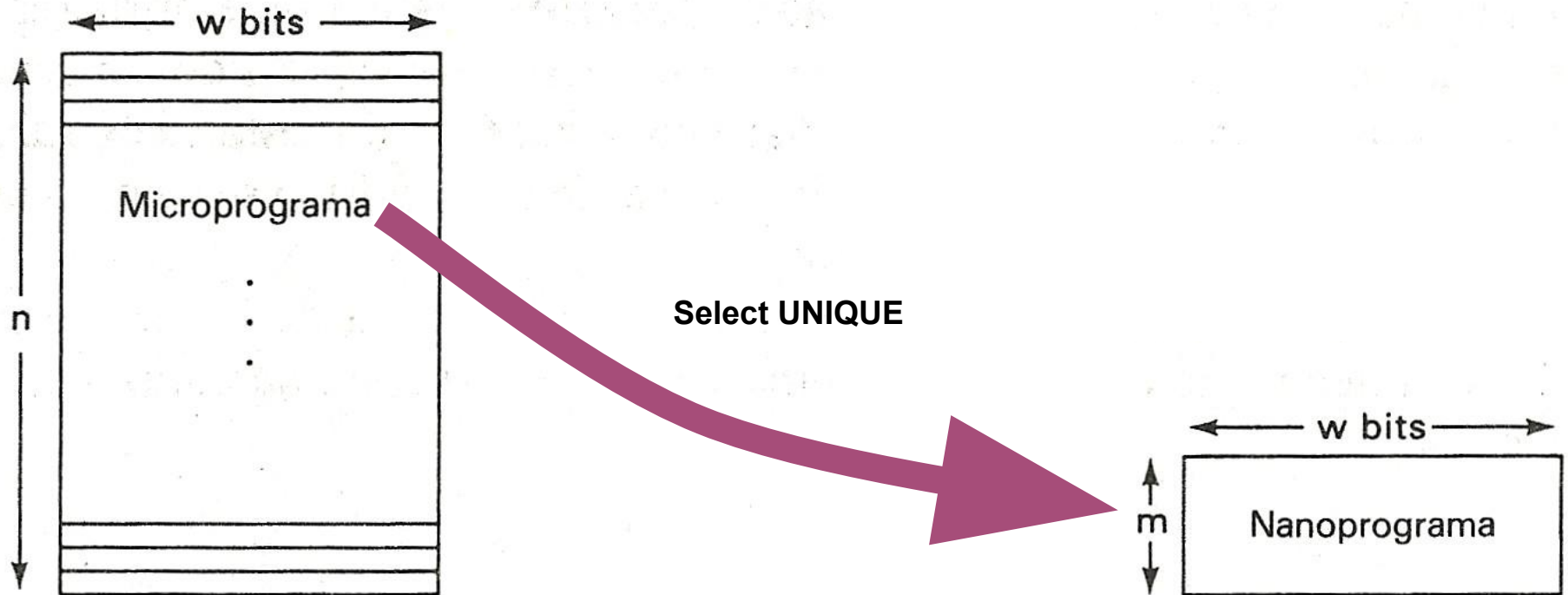
```
   goto 99;
40: tir := lshift(tir);
   if n then goto 46;
41: alu:= tir;
   if n then goto 44;
42: alu:= ac; {JNEG}
   if n then goto 22;
43: goto 0;
44: alu := ac; {JNZE}
   if z then goto 0;
45: pc := ir;
   pc := band (pc, amask);
   goto 0;
46: tir := lshift(tir);
   if n then goto 50;
47: sp := sp + (-1); {CALL}
48: mar := sp; mbr:= pc; wr;
   wr;
49: pc := ir;
   pc := band(pc, amask);
   goto 0;
50: tir := lshift (tir );
   if n then goto 65;
51: tir := lshift(tir);
   if n then goto 59;
52: alu := tir;
   if n then goto 56;
53: mar := ac; rd; {PSHI}
   rd;
54: sp := sp + (-1);
55: a := mbr;
   mar := sp; mbr:=a, wr;
   wr;
   goto 0;
56: mar := sp; rd; {POPI}
57: rd;
   sp := sp + 1;
```

```
8: a := mbr;
   mar := ac; mbr := a; wr;
   wr;
   goto 0;
59: alu := tir;
   if n then goto 62;
60: sp := sp + (-1); {PUSH}
61: mar:= sp; mbr :=ac; wr;
   wr;
   goto 0;
62: mar := sp; rd; {POP}
63: rd;
   sp := sp + 1;
64: ac := mbr;
   goto 0;
65: tir := lshift (tir );
   if n then goto 73;
66: alu := tir;
   if n then goto 70;
67: mar := sp; rd; {RETN}
68: rd;
   sp := sp + 1;
69: pc := mbr;
   goto 0;
70: a := ac; {SWAP}
71: ac := sp;
72: sp:= a;
   goto 0;
73: alu := tir;
   if n then goto 76;
74: a := ir; {INSP}
   a := band(a, smask);
75: sp := sp + a;
   goto 0;
76: a := ir; {DESP}
   a := band(a, smask);
77: a := inv(a);
78: a := a + 1;
   sp := sp + a;
   goto 0;
```

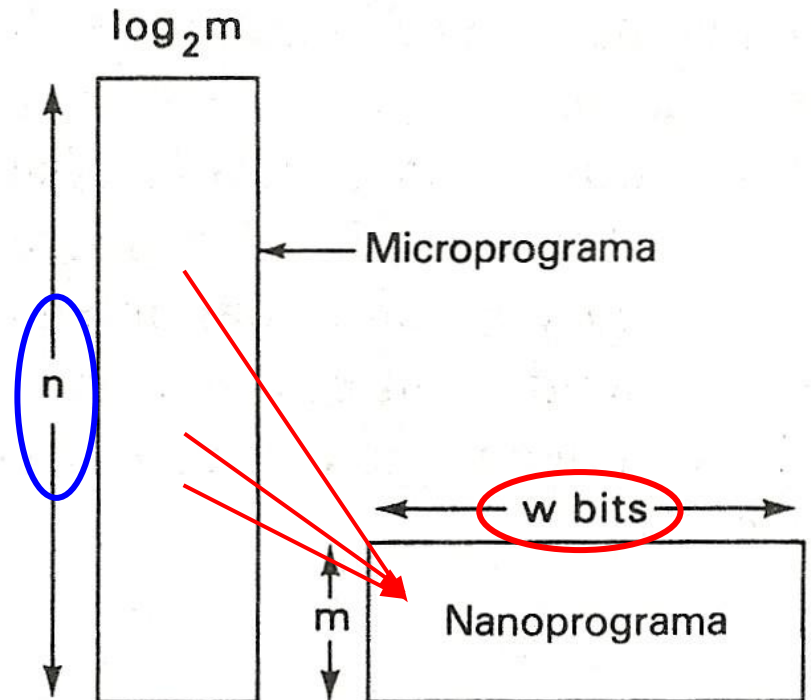
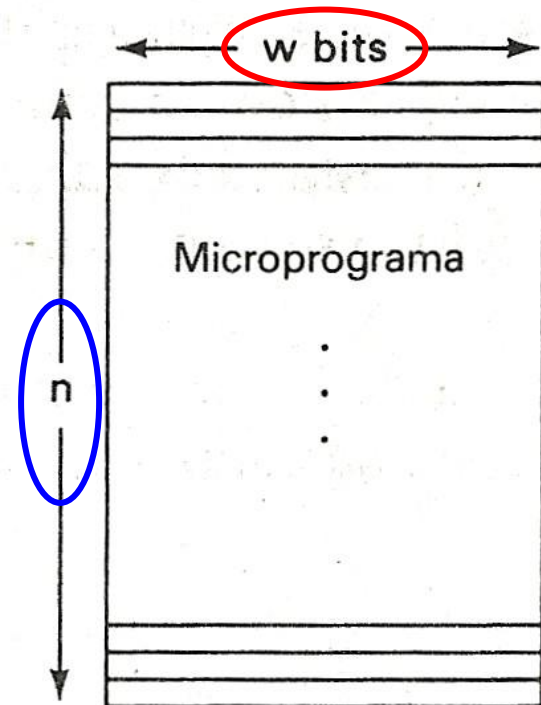
Nano programación



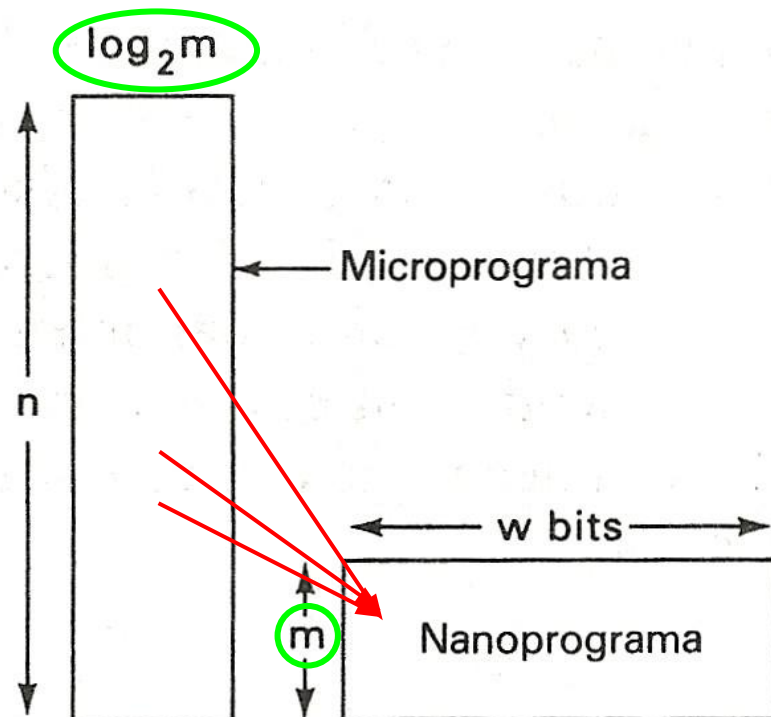
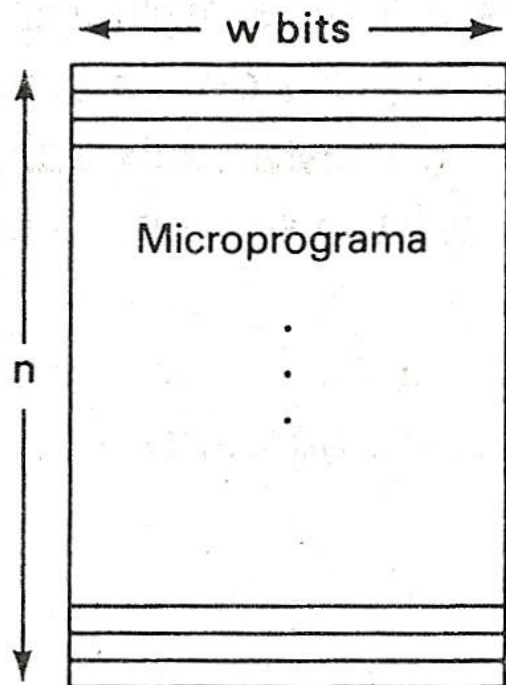
Nano programación



Nano memoria



Nano memoria



Pipe-Line

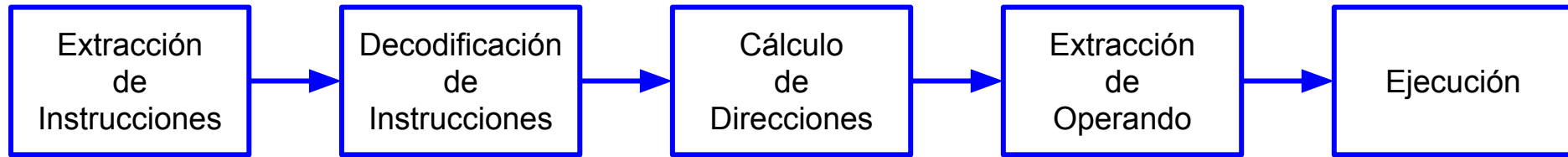
Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta

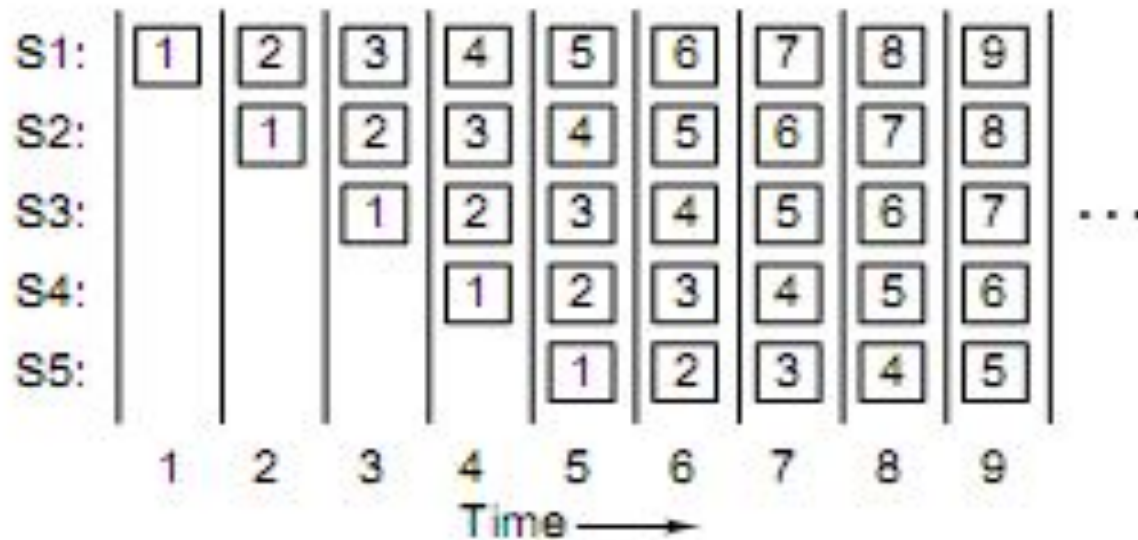
Pipe Line

- Mejora del rendimiento
- Varias unidades específicas

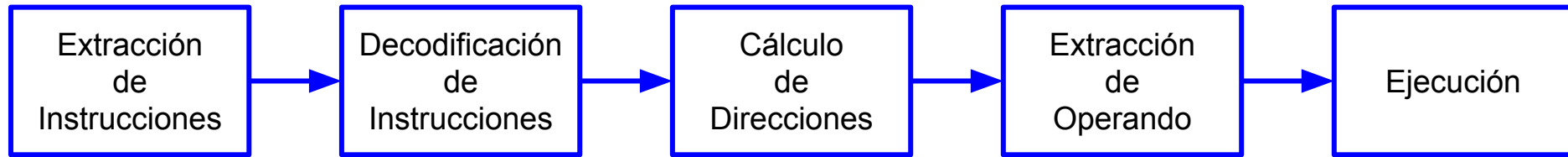
Pipe Line



- Mejora del rendimiento
- Varias unidades específicas

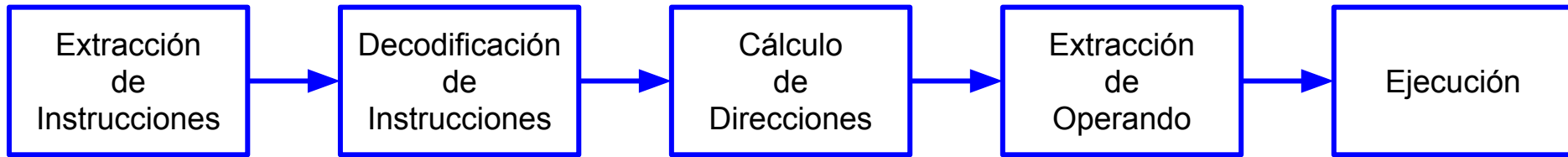


Pipe Line



- Mejora del rendimiento
- Varias unidades específicas
- 30% de las instrucciones son ***saltos***
 - ***incondicionales***
 - ***condicionales***
 - ***iteraciones***

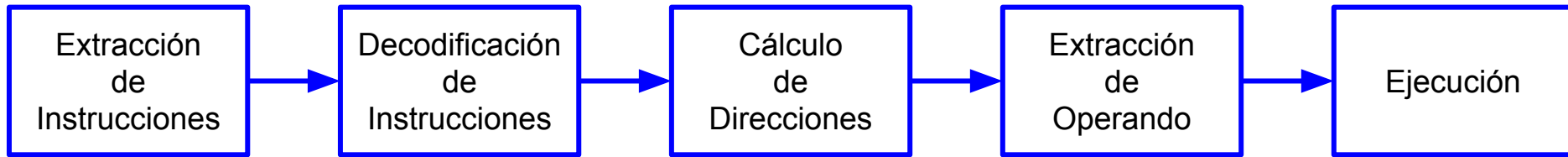
Pipe Line



- Mejora del rendimiento
- Varias unidades específicas
- 30% de las instrucciones son **saltos**

	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

Pipe Line



Los compiladores optimizan el ejecutable a fin de minimizar la Penalización por Salto

	1	2	3	4	5	6	7	8	9	10	11		
Extracción de instrucciones	1	2	B					4	5	6	7	8	9
Decodificación de instrucciones		1	2	B					4	5	6	7	8
Cálculo de direcciones			1	2	B					4	5	6	7
Extracción de operandos				1	2	B					4	5	6
Ejecución					1	2	B					4	5

Micro Arquitectura Vertical

Nano-Programación y Pipe-Line

Arquitectura de computadoras

Pablo A. Montini
Juan I. Iturriaga
Franco Lanzillotta